AIAC Assignment – 11.5

K.Lokesh **2303A51201,B21**

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```python
class Stack:
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)
        print(f"Pushed item: {item}")

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        item = self.stack.pop()
        print(f"Popped item: {item}")
        return item

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.stack) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    print(f"Top item: {stack.peek()}")
    stack.pop()
    print(f"Is stack empty? {stack.is_empty()}")
    stack.pop()
    print(f"Is stack empty? {stack.is_empty()}")
```

OUTPUT:

```
Pushed item: 1
Pushed item: 2
Top item: 2
Popped item: 2
Is stack empty? False
Popped item: 1
Is stack empty? True
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```python
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)
        print(f"Enqueued: {item}")

    def dequeue(self):
        if not self.is_empty():
            item = self.items.pop(0)
            print(f"Dequeued: {item}")
            return item
        else:
            print("Queue is empty. Cannot dequeue.")
            return None

    def peek(self):
        if not self.is_empty():
            print(f"Front of the queue: {self.items[0]}")
            return self.items[0]
        else:
            print("Queue is empty. No front element.")
            return None

    def size(self):
        size = len(self.items)
        print(f"Queue size: {size}")
        return size

    def is_empty(self):
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    queue.peek()
    queue.size()
    queue.dequeue()
    queue.peek()
    queue.size()
    queue.dequeue()
    queue.dequeue()
    queue.dequeue()  # Attempt to dequeue from an empty queue
```

OUTPUT:

```
Enqueued: 1
Enqueued: 2
Enqueued: 3
Front of the queue: 1
Queue size: 3
Dequeued: 1
Front of the queue: 2
Queue size: 2
Dequeued: 2
Dequeued: 3
Queue is empty. Cannot dequeue.
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
# Example usage:
linked_list = LinkedList()
linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)
linked_list.display()  # Output: 10 -> 20 -> 30 -> None
```

OUTPUT:

```
10 -> 20 -> 30 -> None
```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining

    def _hash(self, key):
        return hash(key) % self.size  # Simple hash function

    def insert(self, key, value):
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key-value pair
                print(f"Updated key: {key} with value: {value} at index: {index}")
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))
        print(f"Inserted key: {key} with value: {value} at index: {index}")

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                print(f"Found key: {key} with value: {v} at index: {index}")
                return v
        print(f"Key: {key} not found in the hash table.")
        return None

    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                print(f"Deleted key: {key} from index: {index}")
                return
        print(f"Key: {key} not found in the hash table. Cannot delete.")
# Example usage
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
hash_table.insert("name", "Bob")  # Update existing key
hash_table.search("name")
hash_table.search("age")
hash_table.delete("name")
hash_table.search("name")
hash_table.delete("name")  # Attempt to delete a non-existent key
hash_table.delete("age")
hash_table.search("age")
```

OUTPUT:

```
Inserted key: name with value: Alice at index: 1
Inserted key: age with value: 30 at index: 3
Updated key: name with value: Bob at index: 1
Found key: name with value: Bob at index: 1
Found key: age with value: 30 at index: 3
Deleted key: name from index: 1
Key: name not found in the hash table.
Key: name not found in the hash table. Cannot delete.
Deleted key: age from index: 3
Key: age not found in the hash table.
```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

    pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        print(f"Adding vertex: {vertex}")
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        print(f"Adding edge from {vertex1} to {vertex2}")
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)  # For undirected graph

    def display_connections(self):
        print("Displaying graph connections:")
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {', '.join(edges)}")
# Example usage:
graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "C")
graph.display_connections()
```

OUPUT:

```
Adding vertex: A
Adding vertex: B
Adding vertex: C
Adding edge from A to B
Adding edge from A to C
Adding edge from B to C
Displaying graph connections:
A: B, C
B: A, C
C: A, B
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue
    - Priority Queue
    - Linked List
    - Binary Search Tree (BST)
    - Graph
    - Hash Table
    - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
feature_data_structure_mapping = {
    "Patient Check-In System": ("Queue", "Patients are treated in order of arrival, making a queue the ideal data structure."),
    "Emergency Case Handling": ("Priority Queue", "Critical patients must be treated first, which is best handled by a priority queue."),
    "Medical Records Storage": ("Hash Table", "Fast retrieval of patient details using ID is efficiently managed by a hash table."),
    "Doctor Appointment Scheduling": ("Binary Search Tree (BST)", "Appointments sorted by time can be efficiently managed with a BST for quick insertion and retrieval."
    "Hospital Room Navigation": ("Graph", "Connections between wards and rooms can be represented as a graph for efficient navigation.")
}
# Display the mapping in a table format
print("Feature → Chosen Data Structure → Justification")
for feature, (data_structure, justification) in feature_data_structure_mapping.items():
    print(f"{feature} → {data_structure} → {justification}")
# Implementing the Patient Check-In System using a Queue
class PatientCheckInSystem:
    """A simple patient check-in system using a queue."""

    def __init__(self):
        """Initialize an empty queue for patients."""
        self.queue = []

    def check_in_patient(self, patient_name):
        """Add a patient to the check-in queue."""
        self.queue.append(patient_name)
        print(f"Patient '{patient_name}' checked in.")

    def treat_next_patient(self):
        """Treat the next patient in the queue."""
        if self.queue:
            next_patient = self.queue.pop(0)
            print(f"Treating patient '{next_patient}'.")
        else:
            print("No patients in the queue to treat.")

    def display_queue(self):
        """Display the current queue of patients."""
        if self.queue:
            print("Current patient queue:", ", ".join(self.queue))
        else:
            print("The patient queue is currently empty.")
# Example usage
if __name__ == "__main__":
    check_in_system = PatientCheckInSystem()
    check_in_system.check_in_patient("Alice")
    check_in_system.check_in_patient("Bob")
    check_in_system.display_queue()
    check_in_system.treat_next_patient()
    check_in_system.display_queue()
    check_in_system.treat_next_patient()
    check_in_system.treat_next_patient()
    check_in_system.display_queue()
```

OUTPUT:

```
Feature → Chosen Data Structure → Justification
Patient Check-In System → Queue → Patients are treated in order of arrival, making a queue the ideal data structure.
Emergency Case Handling → Priority Queue → Critical patients must be treated first, which is best handled by a priority queue.
Medical Records Storage → Hash Table → Fast retrieval of patient details using ID is efficiently managed by a hash table.
Doctor Appointment Scheduling → Binary Search Tree (BST) → Appointments sorted by time can be efficiently managed with a BST for quick insertion and retrieval.
Hospital Room Navigation → Graph → Connections between wards and rooms can be represented as a graph for efficient navigation.
Patient 'Alice' checked in.
Patient 'Bob' checked in.
Current patient queue: Alice, Bob
Treating patient 'Alice'.
Current patient queue: Bob
Treating patient 'Bob'.
No patients in the queue to treat.
The patient queue is currently empty.
```

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue

- Priority Queue
- Linked List
- Binary Search Tree (BST)
- Graph
- Hash Table
- Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
class TrafficSignalQueue:
    """A class to represent a traffic signal queue for managing vehicles."""

    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def enqueue(self, vehicle):
        """Add a vehicle to the end of the queue."""
        self.queue.append(vehicle)
        print(f"Vehicle {vehicle} added to the traffic signal queue.")

    def dequeue(self):
        """Remove and return the vehicle at the front of the queue."""
        if not self.is_empty():
            vehicle = self.queue.pop(0)
            print(f"Vehicle {vehicle} removed from the traffic signal queue.")
            return vehicle
        else:
            print("The traffic signal queue is empty.")
            return None

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0

    def size(self):
        """Return the number of vehicles in the queue."""
        return len(self.queue)
# Example usage of the TrafficSignalQueue
if __name__ == "__main__":
    traffic_queue = TrafficSignalQueue()
    traffic_queue.enqueue("Car A")
    traffic_queue.enqueue("Car B")
    traffic_queue.enqueue("Car C")
    print(f"Current queue size: {traffic_queue.size()}")
    traffic_queue.dequeue()
    print(f"Current queue size: {traffic_queue.size()}")
```

OUTPUT:

```
Feature → Chosen Data Structure → Justification
Traffic Signal Queue → Queue → A queue is ideal for managing vehicles waiting at traffic signals as it follows the First-In-First-Out (FIFO) principle, ensuring fair and orderly processing of vehicles.
Emergency Vehicle Priority Handling → Priority Queue → A priority queue allows for the prioritization of emergency vehicles, ensuring they are processed before regular traffic based on their urgency.
Vehicle Registration Lookup → Hash Table → A hash table provides efficient key-value pair storage, allowing for quick access to vehicle details based on registration numbers.
Road Network Mapping → Graph → A graph is suitable for representing roads and intersections as it can model complex relationships and connections between different nodes (intersections).
Parking Slot Availability → Linked List → A linked list can efficiently manage the dynamic nature of parking slot availability, allowing for easy addition and removal of slots.
Vehicle Car A added to the traffic signal queue.
Vehicle Car B added to the traffic signal queue.
Vehicle Car C added to the traffic signal queue.
Current queue size: 3
Vehicle Car A removed from the traffic signal queue.
Current queue size: 2
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue
    - Priority Queue
    - Linked List
    - Binary Search Tree (BST)
    - Graph
    - Hash Table
    - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
class OrderProcessingSystem:
    """A simple order processing system using a queue."""

    def __init__(self):
        """Initialize an empty queue for orders."""
        self.orders = []

    def place_order(self, order):
        """Place a new order in the queue."""
        self.orders.append(order)
        print(f"Order placed: {order}")

    def process_order(self):
        """Process the next order in the queue."""
        if not self.orders:
            print("No orders to process.")
            return None
        order = self.orders.pop(0)  # Dequeue the first order
        print(f"Order processed: {order}")
        return order
# Example usage of the Order Processing System
if __name__ == "__main__":
    order_system = OrderProcessingSystem()
    order_system.place_order("Order #1: Laptop")
    order_system.place_order("Order #2: Smartphone")
    order_system.process_order()  # Should process Order #1
    order_system.process_order()  # Should process Order #2
    order_system.process_order()  # Should indicate no orders to process
```

OUTPUT:

```
Feature → Chosen Data Structure → Justification
Shopping Cart Management → Linked List → A Linked List allows for dynamic addition and removal of products without needing to shift elements, making it ideal for managing a shopping cart.
Order Processing System → Queue → A Queue follows the First-In-First-Out (FIFO) principle, ensuring that orders are processed in the order they were placed.
Top-Selling Products Tracker → Priority Queue → A Priority Queue can efficiently keep track of products based on their sales count, allowing for quick retrieval of top-selling products.
Product Search Engine → Hash Table → A Hash Table provides constant average time complexity for lookups, making it perfect for fast retrieval of products using product IDs.
Delivery Route Planning → Graph → A Graph can represent the connections between warehouses and delivery locations, allowing for efficient route planning and optimization.
Order placed: Order #1: Laptop
Order placed: Order #2: Smartphone
Order processed: Order #1: Laptop
Order processed: Order #2: Smartphone
No orders to process.
```